

Jasmin 도구체인을 이용한 NTT 구현의 안전성 및 상수 시간 검증 방법론 분석

익명 제출

초록

저수준 NTT 구현의 신뢰성은 산술식의 옳고 그름만으로 정해지지 않는다. 루틴이 정의된 방식으로 실행되는지, 그리고 분기와 주소 계산이 비밀 계수에 끌려가지 않는지도 함께 확인되어야 한다. 이 글은 HAETAE 스칼라 NTT 구현을 대상으로 Jasmin 도구체인이 제공하는 안전성 검사와 상수 시간 검사를 어떻게 해석해야 하는지 분석한다. 핵심은 검사기가 읽을 수 있는 형태로 소스를 구성하는 데 있다. 반복 범위, 테이블 오프셋, 포인터 형상, 주소식이 모두 소스 수준에서 드러나야 한다. 그 위에서 `jasminc -checksafety`는 정의된 실행을, `jasmin-ct`는 분기와 주소에 관한 일반 상수 시간성을, `jasmin-ct -speculative`는 Jasmin SCT 모델의 Spectre-v1식 조건을 검사한다. 마지막으로 같은 소스에서 어셈블리를 다시 생성하여 `hpoly.s`와 바이트 단위로 비교한다. 따라서 여기서 얻는 결론은 좁지만 재현 가능하다. 타입 포인터 전제가 만족될 때, 공개된 스칼라 NTT 루틴은 배열 경계 안에서 실행되고, 다항식 계수와 독립적인 공개 제어 흐름 및 주소 흔적을 갖는 것으로 검사된다. 이는 기능 정확성 증명을 보완하는 구현 계층의 증거이지, NTT의 대수적 정확성이나 전체 HAETAE 보안성, 또는 모든 물리적, 미세구조적 누출 부재를 뜻하지 않는다.

1 서론

NTT 커널은 짧은 코드로 구현되지만, 보안상 중요한 조건은 코드의 여러 곳에 흩어져 있다. 공개 일정으로 움직여야 할 메모리 접근이 반복 변수 하나의 잘못된 사용으로 비밀 계수에 의존할 수 있고, 수학적으로는 당연한 테이블 경계도 검사기에는 명시되지 않을 수 있다. 구조상 상수 시간으로 보이는 래퍼 역시 추측 실행 상태에서 넘어온 포인터를 다룰 때에는 별도의 경계 처리가 필요하다. 그러므로 기능 정확성을 논하기 전에, 컴파일될 루틴이 먼저 정의된 의미론 안에서 실행되고 의도한 시간 규율을 따르는지 확인해야 한다.

이 글은 Jasmin으로 작성된 HAETAE 스칼라 NTT 산출물의 구현 수준 성질을 다룬다 [1]. 분석은 추상 예제가 아니라 `jasmin/` 아래의 실제 파일을 대상으로 한다. 공개 진입점은 `hpoly.jazz`에 있고, 단계별 NTT, 역 NTT, 기본 곱셈 커널은 `poly.jinc`에 모여 있다. Montgomery 곱셈은 `reduce.jinc`가 맡고, 파라미터와 트위들 테이블은 각각 `params.jinc`와 `zetass.jinc`에 있다. 검증 절차는 2026년 Jasmin 프레임워크가 현재 기준으로 제시하는 흐름, 곧 소스 안전성 검사, 일반 상수 시간 검사, 추측 실행 상수 시간 검사, 예측 가능한 어셈블리 생성의 순서를 따른다 [1, 2, 7].

여기서 분석하는 것은 전체 구현 정확성이 아니다. NTT가 추상 변환을 계산한다는 사실은 EasyCrypt와 같은 별도 기능 증명의 대상이며, 물리적 부채널 전체를 포괄하는 보안성도 이 글의 범위 밖이다. 질문은 그보다 앞선 구현 계층에

있다. 어떤 소스 구조가 Jasmin 자동 검사기에 충분한 정보를 주는가? 안전성, 일반 상수 시간, 추측 실행 상수 시간 결과는 어떤 전제 아래에서 어셈블리 산출물에 대한 주장으로 읽힐 수 있는가?

1.1 이론적 기초

이 절의 출발점은 하나의 Jasmin 소스에서 얻는 네 가지 조건부 명제이다. 먼저 의미론적 정의성을 확인한다. 공개 루틴이 포인터 전제를 만족하는 인자로 호출되면 실행은 Jasmin 의미론 안에서 끝나야 한다. 다음으로 일반 상수 시간 비간섭성을 확인한다. 공개 입력이 같고 비밀 계수만 다른 두 실행은 같은 분기, 주소 흔적을 가져야 한다. 셋째, 같은 소스를 Jasmin SCT 모델에 넣어 Spectre-v1식 일시 실행 규율을 검사한다. 이때 포인터 보안 수준, 값 보안 수준, 오예측 상태가 따로 추적된다. 마지막 명제는 산출물 대응이다. 검사된 소스를 다시 컴파일한 결과가 제출된 `hpoly.s`와 바이트 단위로 일치해야 한다. 이 네 명제는 NTT의 대수적 기능 정확성이나 HAETAE의 암호학적 안전성보다 약한 구현 계층 명제이며, 서로 대체되지 않는다 [7, 6, 5].

안전성은 나머지 논증이 서 있는 바다이다. 호출이 안전하다는 말은 초기 메모리와 인자에서 최종 메모리와 반환값까지 실행이 줄곧 정의되어 있다는 뜻이다. 이 산출물에서 검사해야 할 항목은 고정 반복문의 종료, 배열 인덱스 경계, 유효하고 정렬된 메모리 접근, 원시 산술 연산의 정의성이다. 예컨대 `reg ptr u32[256]`은 검사기에 원소 타입과 256워드 형상을 알려 주므로 `rp[i]` 접근을 다항식 경계 안에서 판단할 수 있게 한다. 다만 그런 타입이 외부 메모리를 할당하거나 임의의 호출자 주소의 유효성을 보증하지는 않는다. 유효성, 할당 크기, 가변성, 정렬은 여전히 ABI 전제이다. 따라서 `jasminc -checksafety`의 성공은 “그 전제가 충족되면 이 소스는 경계 밖 접근이나 정의되지 않은 원시 연산을 하지 않는다”는 조건부 의미론 명제로 읽어야 한다.

일반 상수 시간성은 `jasmin-ct`가 채택한 누출 모델에 대한 비간섭성으로 표현된다. 이 모델에서 관측되는 것은 비밀 데이터가 시간에 영향을 줄 수 있는 위치, 곧 분기 조건, 반복 조건, 메모리 주소, 배열 인덱스이다. 검사기는 각 식에 보안 수준을 부여한다. 공개 값은 제어 흐름과 주소를 정할 수 있지만, 비밀 값은 산술과 저장값으로만 흘러야 하며 관측 가능한 제어나 주소 위치에 도달해서는 안 된다. NTT에서는 이 분리가 자연스럽다. 단계 번호, 반복 카운터, 트위들 테이블 오프셋, 포인터 기준 주소는 공개 일정이고, 로드된 계수와 Montgomery 곱은 비밀 산술 값이다. 계수를 바꾸면 레지스터 내용과 저장 계수는 달라질 수 있다. 그러나 실행되는 분기와 접근 주소는 달라지면 안 된다.

추측 실행 상수 시간은 별도의, 모델에 상대적인 성질이다. 공개 ABI 경계로 들어오는 포인터는 프로세서가 이미 잘못 예측된 경로에 있을 가능성을 고려해 보수적으로 취급한다.

원리	방법론에서의 의미
정의된 실행	<code>-checksafety</code> 는 공개 포인터 전제하에서 종료, 경계, 유효 메모리, 정렬, 산술 정의성을 확인하는 조건부 증거로 읽는다.
일반 CT 비간섭성	비밀 계수는 산술 값에 영향을 줄 수 있지만, 분기 조건, 반복 조건, 주소, 인덱스는 공개 값이어야 한다.
SCT 모델 분리	추측 실행 결과는 Jasmin의 Spectre-v1/선택적 SLH 검사기 모델에 한정되며, 전체 미세구조 누출 부재로 쓰지 않는다.
타입 포인터 규율	포인터 타입은 원소 형상과 인덱스 경계를 제공하지만, 호출자 주소의 유효성과 정렬은 전제로 남는다.
소스-어셈블리 대응	재구성, 해시 비교, diff 비교로 소스 검사 증거를 <code>hpoly.s</code> 에 연결한다.
주장 분리	안전성, 일반 CT, SCT, 어셈블리 대응, 기능 정확성, 암호학적 보안성은 서로 다른 명제이다.

표 1: Jasmin NTT 검증 방법론의 직접적인 이론적 기반.

래퍼가 `#init_msf()`를 호출하면 오예측 상태 규율이 초기화되고, 생성된 x86-64 코드의 진입부에는 그에 대응하는 fence가 놓인다. 이후 SCT 검사기는 내부 커널이 공개 포인터 기준 주소로만 접근하고, 그 주소가 가리키는 계수 값은 비밀로 유지됨을 확인한다. 이 결과가 배제하는 것은 이 호출 그래프에서 Jasmin SCT 모델이 표현하는 Spectre-v1식 패턴이다. 캐시 교체 상태, 프리패치, 전력, 전자기파, 주파수 변화, 모든 추측 실행 변종까지 포괄하는 주장은 아니다.

마지막 단계인 소스-어셈블리 연결은 필수적이다. 검사기는 Jasmin 소스를 대상으로 동작하지만, 제출 산출물에는 어셈블리가 포함된다. 따라서 같은 소스를 다시 컴파일하고, 해시와 diff로 `hpoly.s`와 비교한다. 파일이 일치하면 안전성, CT, SCT 증거는 해당 컴파일러 버전과 해당 소스 리비전에 대해 검토 중인 어셈블리로 연결된다. 소스, 컴파일러, 대상 옵션, 주석, 어셈블리 중 하나라도 바뀌면 이 사슬은 끊어지고 모든 증거를 다시 생성해야 한다.

논문의 구성. 논문은 주장 범위에서 증거로 나아간다. 2절은 검증 대상 루틴과 주장하지 않는 범위를 정리한다. 3절과 4절은 검사 의무가 이 NTT 구현에 왜 들어맞는지를 연역 구조와 수학적 CT 논증으로 설명한다. 5절부터 9절까지는 실제 Jasmin 검사 절차와 소스 구조를 다룬다. 10절과 11절은 재현 로그를 기록하고 해석하며, 마지막 논의에서는 어떤 경우에 더 강한 제약이나 추가 증명이 필요한지 밝힌다.

2 검증 목표와 범위

검사 대상 산출물은 세 개의 스칼라 루틴을 공개한다.

```
poly_ntt_jazz, poly_invntt_jazz
poly_basemul_jazz.
```

앞의 두 루틴은 $q = 64513$ 위에서 HAETAE가 사용하는 순방향 및 역방향 다항식 변환을 구현하고 [4], 세 번째 루틴은 NTT 영역에서 계수별 기본 곱셈을 수행한다. 이 검증 절차가 확인하는 것은 다음 구현 수준 성질이다.

다항식 계수는 비밀로, NTT 일정은 공개로 둔다. 공개 데이터에는 선택된 루틴, 반복 범위, 단계 길이, 트위들 테이블 오프셋, 래퍼 경계를 지난 포인터 기준 주소가 포함된다. 비밀 데이터에는 다항식 배열에서 로드한 계수와 그 계수에서 파생된 모든 산술 값이 포함된다. 목표 불변식은 단순하다.

성질	사용한 증거	제외되는 주장
안전성	모든 공개 루틴에 대해 <code>jasminc -checksafety</code> 가 위반을 보고하지 않음	임의 호출자 메모리의 유효성
일반 상수 시간	<code>jasmin-ct</code> 가 소스 주석을 수락함	모든 미세구조 누출 부재
추측 실행 상수 시간	<code>jasmin-ct -speculative</code> 가 <code>transient</code> 포인터 래퍼와 MSF 초기화를 수락함	모든 Spectre 변종 또는 물리 누출 부재
어셈블리 연결	재구성 어셈블리가 <code>hpoly.s</code> 와 해시 및 diff에서 일치함	손으로 수정한 어셈블리의 독립 정당성

표 2: 검증 절차가 뒷받침하는 성질과 경계.

계수를 바꾸면 레지스터 내용과 저장되는 계수는 바뀔 수 있지만, 분기 순서와 메모리 주소 순서는 바뀌면 안 된다.

결과는 보통의 저수준 전제에 의존한다. `reg mut ptr u32[HAETAE_N]` 같은 타입은 검사기에 256워드 형상과 인덱스 규율을 제공하지만, 모든 가능한 호출자 포인터가 유효한 객체임을 증명하지는 않는다. 유효성, 할당, 정렬은 호출자 또는 ABI 전제이다. 안전성 검사는 그 전제하에서 Jasmin 소스가 의미론적 영역을 벗어나지 않음을 확인한다.

3 검증 방법의 연역 구조

S 를 Jasmin 소스 `jasmin/hpoly.jazz`, A 를 제출 어셈블리 `hpoly.s`, F 를 세 공개 루틴의 집합이라고 하자. 각 루틴 $f \in F$ 에 대해 P_f 는 호출자 전제를 뜻한다. 즉 모든 포인터 인자는 소스의 타입 포인터 형상에 맞는, 할당되고 정렬되었으며 충분히 큰 메모리를 가리켜야 한다. $\text{Leak}_{A, \bar{U}\Xi}(S, f, \sigma)$ 는 상태 σ 에서 시작한 안전한 실행의 일반 Jasmin 누출 흔적이며, `jasmin-ct`가 관측하는 제어 흐름 및 주소 사건으로 이루어진다. 두 초기 상태가 루틴 선택, 포인터 기준 주소, 배열 크기, 공개 테이블, 반복 범위, 공개 호출 문맥에서 같으면 $\sigma_0 \equiv_{\text{pub}} \sigma_1$ 라고 쓴다. 다항식 계수는 달라도 된다.

이 논문의 연역 구조는 네 전제에 놓인다.

1. 모든 $f \in F$ 에 대해 $P_f \Rightarrow \text{Safe}(S, f)$.

2. 모든 $f \in F$ 에 대해 $\sigma_0 \equiv_{\text{pub}} \sigma_1$ 이면

$$\text{Leak}_{A, \bar{U}\Xi}(S, f, \sigma_0) = \text{Leak}_{A, \bar{U}\Xi}(S, f, \sigma_1).$$

3. 모든 $f \in F$ 에 대해 $\text{SCT}_{\text{Jasmin}}(S, f)$.

4. 재생성 어셈블리는 $\text{Compile}_{\text{Jasmin}}(S) = A$ 를 만족한다.

따라서 `hpoly.s`는 검사된 소스에서 생성된 제출 어셈블리이며, P_f 아래에서 이 소스 수준 증거는 세 스칼라 NTT 공개 루틴의 안전성, 일반 CT, Jasmin SCT 성질을 뒷받침한다.

첫 전제는 `jasminc -checksafety`, 둘째는 `jasmin-ct`, 셋째는 `jasmin-ct -speculative`, 넷째는 어셈블리 재생성과 `hpoly.s` 비교로 확인된다. 이 규칙은 포인터 전제와 로그에 기록된 고정된 컴파일러/소스 쌍에 의존한다. 또한 구현 수준의 규칙이다. 어떤 전제도 NTT 공식의 대수적 정확성, 기본 곱셈의 기능 정확성, 또는 전체 HAETAE 서명 방식의 암호학적 안전성을 말하지 않는다.

정리 형태의 해석. 로그가 해당 소스 리비전에 대해 최신이고 모든 검사 명령이 성공적으로 종료했다면, P_f 를 만족하는 모든 공개 호출은 정의된 소스 실행을 갖는다. 같은 공개 입력을 갖는 두 실행은 일반 Jasmin 분기/주소 누출 흔적을 갖는다. 또한 공개 래퍼는 **transient** 포인터 입력을 받아 포인터가 가리키는 비밀 값을 보존하는 Jasmin SCT 규율을 만족한다. 또한 제출된 **hpoly.s**는 그 검사된 소스에서 재생성된 어셈블리이다. 반대로 소스, 주석, 컴파일러 버전, 대상 옵션, 어셈블리 파일, 포인터 전제 중 하나가 바뀌면 이 결론은 새 증거 없이 유지되지 않는다.

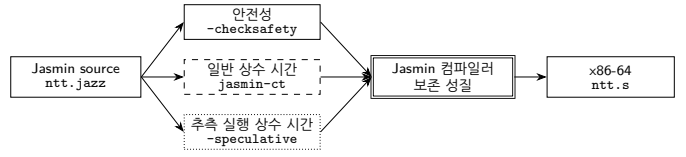


그림 1: 검증 절차.

4 NTT 커널의 수학적 상수 시간 논증

스칼라 NTT 구현의 상수 시간성은 공개 실행 일정에 대한 논증으로 설명된다. $N = 256$, $q = 64513$ 이라고 하자. 다항식은 $\mathbb{Z}_q$ 의 계수 대표 $a = (a_0, \dots, a_{255})$ 로 이루어진 배열이다. 계수 대표는 비밀로, 루틴 이름, 포인터 기준 주소, 반복 범위, 단계 길이, 트위들 테이블 위치는 공개로 취급한다.

대수적으로 각 단계는 $\mathbb{Z}_q$ 위의 고정 선형 변환이다. 순방향 버터플라이는 위치 i, k 에 대해

$$(a_i, a_k) \mapsto (a_i + \zeta_z a_k, a_i - \zeta_z a_k)$$

를 적용한다. 역방향 버터플라이는 공개 역 트위들 테이블과 마지막 공개 스케일 인자를 사용해 이에 대응하는 역방향 단계 연산을 수행한다. 따라서 Jasmin 프로그램의 계산 그래프는 표준 기수 2 NTT 버터플라이 그래프이다. 정점은 계수 위치이고, 간선은 공개 버터플라이 쌍이며, 간선 라벨은 공개 트위들 테이블 원소이다. CT 논증은 바로 이 그래프 성질을 사용한다. 계수 값이 공개일 필요는 없고, 트위들 테이블이 의도한 테이블이라는 대수적 사실 자체를 이 검사로 증명하는 것도 아니다. 그 사실은 기능 정확성 검증의 몫이다. 여기서 확인하는 것은 선택된 테이블이 계수와 무관한 공개 일정으로 사용된다는 점이다.

순방향 단계에서 공개 버터플라이 길이를 $\ell \in \{128, 64, 32, 16, 8, 4, 2, 1\}$ 라 하고 $B = N/(2\ell)$ 라 두자. 해당 Jasmin 보조 함수는 공개 카운터

$$0 \leq b < B, \quad 0 \leq j < \ell$$

에 대해

$$i = b(2\ell) + j, \quad k = i + \ell, \quad z = B + b$$

를 계산한다. 그러므로 $0 \leq i < 256$, $0 \leq k < 256$ 이고, $\text{rp}[i]$, $\text{rp}[k]$, $\text{jzetaz}[z]$ 의 주소는 모두 공개 값의 함수이다. 버터플라이 산술은

$$t = \zeta_z a_k, \quad a_i^+ = a_i + t, \quad a_k^+ = a_i - t$$

형태이며, 곱셈은 `_fqmul`에 구현된 고정 Montgomery 곱셈이다. $a_i, a_k, t, a_i^+, a_k^+$ 는 비밀일 수 있지만, 분기, 반복 범위, 배열 인덱스, 메모리 주소를 정하는 데 쓰이지 않는다.

역방향 단계도 같은 흔적 구조를 갖는다. 공개 길이 $\ell \in \{1, 2, 4, 8, 16, 32, 64, 128\}$ 에 대해 카운터는 다시 $i = b(2\ell) + j$, $k = i + \ell$ 을 정한다. 소스가 사용하는 역방향 단계의 테이블 기준은 $I_\ell = 256 - N/\ell$ 이고, 트위들 인덱스는 $z = I_\ell + b$ 이다. 따라서 역방향 트위들 주소도 블록 카운터의 공개 아핀

함수이다. 마지막 스케일 반복문은 공개 카운터 $0 \leq j < N$ 에 대해 $\text{rp}[j]$ 만 접근한다. 산술은

$$a_i^+ = a_i + a_k, \quad a_k^+ = \zeta_z(a_i - a_k)$$

형태이고, 이후 공개 일정의 스케일링이 이어진다. 여기서도 비밀 계수는 산술 데이터에만 영향을 준다.

기본 곱셈은 더 직접적이다. 공개 $0 \leq i < N$ 에 대해 $\text{ap}[i]$ 와 $\text{bp}[i]$ 를 읽고, `_fqmul(ai, bi)`를 계산한 뒤 $\text{rp}[i]$ 에 쓴다. 접근 위치는 공개 인덱스 i 와 공개 포인터 기준 주소가 결정하고, 로드된 계수와 곱은 비밀 산술 값이다.

결론적으로 공개 입력이 같고 계수만 다른 두 안전한 실행은 순방향 NTT, 역방향 NTT, 기본 곱셈에서 같은 코드 분기결과 같은 메모리 주소열을 방문한다. 이것이 Jasmin CT 주석이 표현하고 `jasmin-ct`가 구문적으로 검사하는 수학적 불변식이다. 따라서 기록된 CT 결과는 단순한 외부 관찰이 아니다. 위 공개 일정 논증을 Jasmin의 기본 분기/주소 누출 모델 안에서 도구가 확인한 것이다.

5 방법론 개요

그림 1은 검증 절차를 요약한다. 절차는 하나의 Jasmin 소스 진입점 `jasmin/hpoly.jazz`에서 시작한다. 이 파일은 커널 코드를 포함하고 세 ABI 수준 래퍼를 공개한다. 모든 검사는 같은 소스 파일에 대해 수행된다. 안전성 전용 소스나 CT 전용 소스를 따로 두지 않는다.

운영 절차는 다섯 단계이다.

1. `jasminc -version`으로 도구체인 버전을 기록한다.
2. `jasminc -checksafety`로 안전성 검사를 실행한다.
3. `jasmin-ct`로 일반 상수 시간 검사를 실행한다.
4. `jasmin-ct -speculative`로 추측 실행 검사를 실행한다.
5. 어셈블리를 다시 생성하고 제출된 `hpoly.s`와 비교한다.

소스, 컴파일러 버전, 대상 옵션, 제출 어셈블리가 바뀌면 모든 증거를 다시 생성해야 한다.

6 Jasmin 소스 준비

자동 검증은 명령어만 실행하는 일이 아니다. 소스가 검사기에 필요한 불변식을 읽을 수 있는 형태로 드러내야 한다. 스칼라 HAETAE NTT의 핵심 선택은 단계 특수화이다. 하나의 반복문에서 단계 길이를 변경하는 대신, 순방향 변환은

$$128, 64, 32, 16, 8, 4, 2, 1$$

파일	산출물 역할	검증상 의미
hpoly.jazz	공개 ABI 래퍼	상수 시간 및 추측 실행 상수 시간 주석과 #init_msf() 경계를 담는다.
poly.jinc	NTT, 역 NTT, 기본 곱셈 커널	단계별 공개 반복문과 공개 인덱스 식을 드러낸다.
reduce.jinc	Montgomery 곱셈 및 환원	계수 산출을 고정된 직선형 보조 코드에 둔다.
params.jinc, zetas.jinc	공개 파라미터와 트위들 테이블	$N = 256$, $q = 64513$, 공개 테이블 주소를 고정한다.
hpoly.s	제출 어셈블리 산출물	재생성 어셈블리와 일치해야 소스 수준 증거가 적용된다.

표 3: 소스 파일과 검증상 역할.

의 고정 길이 보조 함수를 사용하고, 역방향 변환은 그 반대 순서의 보조 함수를 사용한다. 소스 구성은 작지만, 파일마다 검증에서 맡는 역할이 다르다.

첫 순방향 보조 함수는 이 구조를 잘 보여준다.

Listing 1: 단계 특수화된 순방향 보조 함수.

```
fn _poly_ntt_stage_128(reg mut ptr u32[N] rp)
-> reg ptr u32[N] {
    reg ptr u32[256] zetasp;
    reg ui64 block j start idx offset;
    reg u32 zeta t s coeff;

    zetasp = jzetasp;
    block = 0;
    while (block < 1) {
        zeta = zetasp[1 + block];
        start = block;
        start <= 8;
        j = 0;
        while (j < 128) {
            idx = j;
            idx += start;
            s = rp[idx];
            offset = idx;
            offset += 128;
            coeff = rp[offset];
            t = __fqmul(zeta, coeff);
            coeff = s;
            coeff -= t;
            rp[offset] = coeff;
            s += t;
            rp[idx] = s;
            j += 1;
        }
        block += 1;
    }
    return rp;
}
```

Analyzing function poly_ntt_jazz

*** No Safety Violation

Memory ranges:
mem_rp: [0; 0]

* Rel:
{mem_rp = 0}
mem_rp [0; 0]

이 형태는 안전성 검사와 상수 시간 검사 모두에 도움이 된다. 안전성 검사기는 $block < 1$, $j < 128$, $offset = idx + 128$ 같은 리터럴 사실을 본다. 시간 검사기는 모든 주소가 포인터 기준 주소, 공개 카운터, 공개 단계 상수로 계산됨을 본다. 로드된 s 와 $coeff$ 는 비밀이지만 산출과 저장에만 쓰인다.

공개 래퍼는 보안 경계를 담당한다. 예를 들어 순방향 래퍼는 일반 CT와 SCT 검사를 위한 주석을 모두 갖는다.

Listing 2: 공개 래퍼 주석.

의무	소스에서 쓰이는 사실	예
다항식 경계	타입 포인터는 256워드 형상을 갖고 인덱스는 공개이며 경계 안에 있다	$j < 128$ 이면 $j + 128 < 256$
트위들 경계	단계 보조 함수가 테이블 오프셋과 블록 범위를 고정한다	첫 단계는 $block < 1$ 에서 $jzetas[1 + block]$ 를 읽는다
메모리 가변성	출력 포인터는 가변이고 기본 곱셈 입력은 const이다	$rp[i]$ 는 쓰고 $ap[i]$, $bp[i]$ 는 읽는다
산술 정의성	Montgomery 보조 함수는 명시적 캐스트, 부호 있는 shift, 워드 폭을 사용한다	$__fqmul$ 이 Montgomery 환원을 호출한다

표 4: 안전성 의무와 그에 대응하는 소스 사실.

```
#[ct = "rp -> rp"]
#[sct = "{ ptr: transient, val: secret } ->
{ ptr: public, val: secret }"]
export
fn poly_ntt_jazz(reg mut ptr u32[N] rp)
-> reg ptr u32[N] {
    _ = #init_msf();
    rp = _poly_ntt(rp);
    return rp;
}
```

일반 상수 시간 주석은 소스 수준 정보 흐름 정책을 말한다. SCT 주석은 포인터의 보안 수준과 포인터가 가리키는 값의 보안 수준을 분리한다. 래퍼는 ABI 경계에서 transient 포인터를 받고, 오예측 상태를 초기화한 뒤, 계수 값은 비밀로 유지하면서 내부 커널에는 공개 포인터 기준 주소를 넘긴다.

Listing 3: 내부 함수.

```
/* Forward NTT (in-place, bit-reversed output). */
fn _poly_ntt(reg mut ptr u32[N] rp) -> reg ptr u32[N]
{
    rp = _poly_ntt_stage_128(rp);
    rp = _poly_ntt_stage_64(rp);
    rp = _poly_ntt_stage_32(rp);
    rp = _poly_ntt_stage_16(rp);
    rp = _poly_ntt_stage_8(rp);
    rp = _poly_ntt_stage_4(rp);
    rp = _poly_ntt_stage_2(rp);
    rp = _poly_ntt_stage_1(rp);
    return rp;
}
```

7 안전성 검사

안전성 검사는 jasminc로 실행한다.

Listing 4: 안전성 검사 명령.

```
jasminc -checksafety -o ntt.s ntt.jazz
```

기대되는 출력은 poly_ntt_jazz, poly_invntt_jazz, poly_basemul_jazz 각각에 대해 No Safety Violation을 포함한다. 여기서 어셈블리 출력 파일은 부수적이다. 명령의 목적은 컴파일 과정에서 안전성 분석을 수행하는 데 있다.

이 출력은 조건부로 읽어야 한다. 안전성 검사가 성공했다고 해서 호출자가 넘기는 임의의 기계 주소가 유효하다는 뜻은 아니다. 호출자가 타입 포인터 전체를 만족하면, 소스 실행이 선언된 배열 안에 머물고 Jasmin 의미론의 원시 연산을 정의된 방식으로 사용한다는 뜻이다. 이 조건부 해석이 중요하다. 안전성은 기능 논증과 시간 논증이 모두 의존하는 정의성 계층이다.

프로그램 구성요소	기대 라벨	이유
반복 범위와 카운터	공개	제어 흐름 길이를 결정한다
트위들 테이블 오프셋	공개	테이블 주소열을 결정한다
래퍼 이후 포인터 기준 주소	공개	메모리 주소를 결정한다
로드된 다항식 계수	비밀	입력 의존 데이터를 담는다
Montgomery 곱과 합	비밀	계수에서 파생된다

표 5: 일반 상수 시간 라벨링 규율.

8 일반 상수 시간 검사

일반 상수 시간 검사는 다음 명령으로 실행한다.

Listing 5: 일반 상수 시간 검사 명령.

```
jasmin-ct ntt.jazz
```

이 산출물에서는 성공 시 진단 출력이 없다. 이 침묵은 수동 점검이 아니라, 검사기가 공개 주석 아래에서 프로그램을 순회하고 관측 위치에 도달하는 식이 공개임을 확인했다는 뜻이다.

이 누출 모델은 분기 조건, 반복 조건, 메모리 주소, 배열 인덱스를 관측 대상으로 둔다. 따라서 방법론은 다음 라벨링 규율을 요구한다.

기본 곱셈 커널은 가장 단순한 예이다. 공개 256회 반복에서 `ap[i]`와 `bp[i]`를 읽고 `__fcmul(a,b)`를 계산한 뒤 `rp[i]`에 쓴다. 값 `a`, `b`, 곱은 비밀이다. 인덱스 `i`와 접근 주소는 공개이다. NTT 단계도 중첩된 공개 일정한 더해질 뿐 같은 규율을 따른다.

9 추측 실행 상수 시간 검사

추측 실행 검사는 다음과 같이 실행한다.

Listing 6: 추측 실행 검사 명령.

```
jasmin-ct --speculative ntt.jazz
```

검사기는 내부 루틴과 공개 루틴에 대해 추론한 보안 타입을 출력한다. 공개 NTT 래퍼에서 중요한 형태는 다음과 같다.

```
{ptr:transient, val:secret}
→
{ptr:public, val:secret}.
```

`poly_basemul_jazz`의 세 포인터 인자도 같은 `transient` 포인터, `secret-value` 입력 형태를 가지며, 결과는 비밀 값을 가리키는 공개 포인터이다.

핵심 소스 연산은 목록 3의 `#init_msf()`이다. 이 소스에서 생성된 x86-64 어셈블리를 보면 각 공개 래퍼가 `lfence`로 시작한다. 따라서 공개 진입 경계 자체가 증거의 일부가 된다. ABI에서 넘어온 포인터 기준 주소는 추측 실행 상태가 정리되기 전까지 공개 주소로 간주하지 않고, 그 뒤 내부 커널에서만 공개 카운터와 함께 주소 계산에 사용한다. 계수 값은 이 과정 내내 비밀로 남는다.

이 결과는 모델에 상대적이다. 해당 소스와 호출 그래프가 Jasmin SCT/Spectre-v1식 검사기 모델을 만족한다는 뜻이지, 전력, 전자기파, 주파수 조절, 캐시 교체 상태, 모든 추측 실행 변종을 실험적으로 배제한다는 뜻은 아니다.

로그	해석
jasminc-version.log	Jasmin 컴파일러 버전을 기록한다.
jasmin-checksafety.log	세 공개 루틴 모두에서 안전성 위반이 없음을 보고한다.
jasmin-ct.log	일반 상수 시간 검사가 성공하면 비어 있거나 공백만 있을 수 있다.
jasmin-ct-speculative.log	공개 래퍼의 <code>transient</code> 포인터와 내부 공개 포인터/비밀 값 구조를 보여준다.
jasmin-compile.log	어셈블리 생성 성공 시 비어 있을 수 있다.
assembly-sha256.log, assembly-diff.log	재구성 어셈블리가 <code>hpoly.s</code> 와 일치함을 보인다.

표 6: 재현성 증거의 해석.

10 소스 검사와 어셈블리의 연결

소스 수준 검사가 제출 어셈블리에 의미를 가지려면, 그 어셈블리가 검사된 소스의 컴파일 결과여야 한다. 따라서 검증 절차에는 어셈블리 재생성과 비교가 포함된다.

Listing 7: 어셈블리 재생성과 비교.

```
jasminc jasmin/hpoly.jazz -o /tmp/hpoly-after.s
sha256sum hpoly.s /tmp/hpoly-after.s
diff -u hpoly.s /tmp/hpoly-after.s
```

기대 결과는 해시가 같고 `diff`가 비어 있는 것이다. 이 단계는 별도의 바이너리 분석 증명을 만들지 않는다. 안전성과 시간 검사를 통과한 같은 Jasmin 소스가 검토 중인 어셈블리를 실제로 산출했다는 연결만 제공한다. `hpoly.s`를 손으로 수정했거나, 다른 도구체인으로 컴파일했거나, 대상 옵션을 바꾸었다면 소스-어셈블리 증거를 다시 만들어야 한다.

11 재현성 증거

검증 절차는 감사 가능해야 한다. 완전한 재현 기록에는 도구 버전, 소스 리비전, 검사 로그, 컴파일 로그, 어셈블리 비교가 포함되어야 한다. 본 산출물에서 사용한 증거는 다음 명령 열로 갱신할 수 있다.

Listing 8: 재현 로그 갱신 명령.

```
jasminc -version > paper/logs/jasminc-version.log
git rev-parse --short HEAD > paper/logs/git-revision.log
jasminc -checksafety -o /tmp/hpoly-safe.s jasmin/hpoly.jazz \
> paper/logs/jasmin-checksafety.log 2>&1
jasmin-ct jasmin/hpoly.jazz > paper/logs/jasmin-ct.log 2>&1
jasmin-ct --speculative jasmin/hpoly.jazz \
> paper/logs/jasmin-ct-speculative.log 2>&1
jasminc jasmin/hpoly.jazz -o /tmp/hpoly-after.s \
> paper/logs/jasmin-compile.log 2>&1
sha256sum hpoly.s /tmp/hpoly-after.s > paper/logs/assembly-sha256.log
diff -u hpoly.s /tmp/hpoly-after.s > paper/logs/assembly-diff.log
```

리다이렉트된 로그가 비어 있거나 공백뿐이어도 명령의 종료 상태는 증거의 일부이다. 특히 `jasmin-ct.log`가 비어 있거나 출바꿈만 있거나, `jasmin-compile.log` 또는 `assembly-diff.log`가 비어 있는 것은 종료 상태 0과 함께 읽을 때만 성공의 증거가 된다. 앞으로의 보존용 실행은 각 명령의 상태 파일을 별도로 남기면 더 명시적이다. 그러나 이 논문의 논리적 주장은 언제나 파일 내용과 성공적인 명령 종료로 함께 전제한다.

12 검증 결과 분석

기록된 로그를 통해 이 절차는 단순한 명령 목록이 아니라 감사 가능한 산출물 주장으로 바뀐다. 실행 환경은 `jasminc-version.log`와 `git-revision.log`에 남아 있다. 사용한 도구는 Jasmin 컴파일러 2026.03.0이고, 소스 리버전은 `f0e1399`이다. 소스와 검사기 구현이 모두 주장의 입력이므로 이 두 값도 결과의 일부이다. `jasmin-ct.log`에는 줄바꿈만 있고, `jasmin-compile.log`와 `assembly-diff.log`는 0바이트이다. `assembly-sha256.log`는 `hpoly.s`와 `/tmp/hpoly-after.s`의 SHA-256 해시 값이 같다고 기록한다.

안전성 로그가 먼저 보아야 할 자료이다. 이 로그는 세 공개 함수를 따로 분석하고 `poly_ntt_jazz`, `poly_invntt_jazz`, `poly_basemul_jazz` 각각에 대해 No Safety Violation을 출력한다. 두 in-place NTT 루틴에는 상정 메모리 이름으로 `mem_rp`만 나타나고, 기본 곱셈에는 `mem_rp`, `mem_ap`, `mem_bp`가 나타난다. 세 경우 모두 범위는 `[0,0]`이고 `mem_rp = 0` 같은 등식 관계가 붙어 있다. 이 부분은 조심해서 해석해야 한다. 검사기가 이 고정 크기 타입 포인터 소스에 대해 별도의 가변 길이 메모리 영역 의무를 만들지 않았다는 뜻이지, 임의의 기계 주소가 안전하다는 뜻은 아니다. 결과는 여전히 공개 타입 포인터 인자가 유효하고 적절히 정렬된 호출자 메모리를 가리킨다는 전제에 의존한다.

일반 상수 시간 로그에는 줄바꿈만 있다. 이 사실은 명령이 성공적으로 종료했다는 사실과 함께 읽을 때 의미가 있다. `jasmin-ct`가 공개 주석에 대해 실패한 타이핑 의무를 보고하지 않았다는 뜻이기 때문이다. 따라서 Jasmin CT 타입 시스템 모델 안에서는 계수 값이 비밀로 남아도, 관측되는 제어 흐름과 메모리 주소식은 공개로 판정된다. 이는 공개 일정과 비밀 다항식 계수를 분리한 NTT 구조와 부합한다.

추측 실행 상수 시간 로그는 더 길며 공개 경계를 확인하는 데 유용하다. 내부 단계 보조 함수들은 포인터 성분이 공개이고 값 성분이 비밀인 `modmsf` 루틴으로 기록된다. 세 공개 래퍼는 `transient` 포인터 입력과 비밀 값을 가리키는 공개 포인터 출력을 갖는다. 예를 들어 `poly_ntt_jazz`와 `poly_invntt_jazz`는 `{ptr : transient, val : secret} → {ptr : public, val : secret}` 형태이고, `poly_basemul_jazz`는 세 포인터 인자 모두에 같은 입력 규율을 적용한다. 이는 `#init_msf`가 정당화하는 래퍼 패턴과 정확히 맞다. ABI에서 들어오는 포인터는 경계에서 보수적으로 취급되고, 그 뒤 내부 커널은 공개 포인터 기준 주소와 비밀 계수 값으로 동작한다.

나머지 `jasminc` 증거는 소스 수준 결과를 제출 어셈블리에 연결한다. `jasmin-compile.log`는 비어 있어 진단 없이 어셈블리 재생성이 성공했음을 기록한다. `assembly-diff.log`도 비어 있고, `assembly-sha256.log`는 `hpoly.s`와 `/tmp/hpoly-after.s`의 해시가 같음을 보인다. 따라서 검사된 소스와 제출 어셈블리는 이 도구체인 실행에서 바이트 단위로 연결된다.

`hpoly.jazz`, `poly.jinc`, `reduce.jinc`, `params.jinc`, `zetas.jinc`, `hpoly.s`, Jasmin 컴파일러 버전, 대상 옵션이 바뀌면 검사를 다시 실행해야 한다. 원시 메모리 연산, 새 공개 루틴, 변경된 상수 시간 또는 추측 실행 상수 시간 주석, 데이터 의존 분기, 계수 인덱스 테이블, 비밀 해제 (declassification)가 추가되는 패치도 별도로 검토해야 한다.

증거	관측된 로그 결과	분석상 결론
도구와 소스 버전	Jasmin Compiler 2026.03.0; 리버전 <code>f0e1399</code>	주장은 구체적 검사기/컴파일러와 소스 스냅샷에 묶인다.
안전성	세 공개 함수가 No Safety Violation을 보고	타입 포인터 호출자 전제하에서 소스 실행이 정의된다.
일반 상수 시간	성공 실행 뒤 <code>jasmin-ct.log</code> 가 공백뿐임	공개 주석에 대해 일반 CT 타이핑 의무가 실패하지 않았다.
추측 실행 상수 시간	공개 래퍼가 <code>transient</code> 포인터를 공개 포인터와 비밀 값으로 타이핑	<code>#init_msf</code> 기반 SCT 경계와 일치한다.
어셈블리 재생성	컴파일 및 diff 로그가 비어 있고 해시가 일치	<code>hpoly.s</code> 는 검사된 소스에서 재생성된 어셈블리이다.

표 7: 기록된 검증 로그의 분석.

13 논의와 한계

스칼라 NTT는 구조가 규칙적이어서 이 검증 절차와 잘 맞는다. 단계별 특수화 보조 함수는 안전성 검사기가 필요한 작은 정수 사실을 소스에 드러낸다. CT 논증은 프로그래머의 직관적 규율과 일치한다. 카운터와 테이블 오프셋은 공개이고, 계수와 파생 산술 값은 비밀이다. 공개 래퍼는 하나의 분명한 추측 실행 경계를 제공하며, 내부 커널은 추가 `transient` 포인터 경계를 만들지 않는다.

반대로 데이터 의존 종로 조건, 복잡한 원시 포인터 산술, 복잡한 힙 구조, 비밀 해제 (declassification)가 있는 코드에 같은 절차를 기계적으로 적용하면 결과 해석이 부족할 수 있다. 이런 경우에는 통과 결과에 대한 추가 설명, 계약, 수동 증명이 필요할 수 있다. 2026년 Jasmin 자료는 더 큰 프로그램을 위한 새로운 모듈식 안전성 검사 방향을 설명한다 [7]. 이 산출물은 상징적 힙 할당, 소스 스택 배열, 비밀 의존 반복 범위가 없는 스칼라 NTT이므로 `jasminc -checksafety`를 사용한다.

또한 이 검증 절차는 Jasmin에 대한 최근 컴파일러 보안 연구가 다루는 암호학적 보안성 보존과도 구별된다 [3]. 그 연구는 더 높은 수준의 암호학적 보안 성질이 컴파일러 전달을 거치며 어떻게 보존되는지를 다룬다. 여기서 제시한 절차는 그보다 낮은 구현 계층, 즉 정의된 실행, 공개 분기/주소 흔적, 소스-어셈블리 대응을 제공한다.

14 결론

이 글은 Jasmin으로 작성된 NTT 구현에서 안전성과 상수 시간성을 어떻게 확인할 수 있는지 분석했다. 핵심은 세 가지이다. 첫째, 검사 의무가 소스에 드러나도록 공개 일정, 포인터 형상, 주소식을 구성한다. 둘째, 같은 공개 소스에 대해 안전성, 일반 상수 시간, 추측 실행 상수 시간 검사 결과를 각각의 모델과 전제에 맞게 해석한다. 셋째, 재생성과 비교를 통해 그 소스 수준 결과를 제출 어셈블리에 연결한다. HAETAE 스칼라 NTT에 이 절차를 적용하면, 타입 포인터 전제하에서 공개 스칼라 루틴이 정의된 방식으로 실행되고 Jasmin 검사기 모델 안에서 비밀 다항식 계수와 무관한 분기 및 메모리 주소 흔적을 갖는다는 재현 가능한 산출물 주장을 얻는다. 이 주장은 기능적 증명과 상위 암호학적 논증을 생성된 저수준 코드에 연결하기 위해 먼저 확보해야 할 구현 계층의 근거이다.

참고문헌

- [1] J. B. Almeida, M. Barbosa, G. Barthe, A. Blot, B. Gregoire, V. Laporte, T. Oliveira, H. Pacheco, B. Schmidt, and P.-Y. Strub. Jasmin: High-assurance and high-speed cryptography. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1807–1823. ACM, 2017.
- [2] J. B. Almeida, M. Barbosa, G. Barthe, B. Gregoire, A. Koutsos, V. Laporte, T. Oliveira, and P.-Y. Strub. The last mile: High-assurance and high-speed cryptographic implementations. In *2020 IEEE Symposium on Security and Privacy*. IEEE, 2020.
- [3] S. Arranz-Olmos, G. Barthe, L. Blatter, B. Gregoire, V. Laporte, and P. Torrini. The jasmin compiler preserves cryptographic security: Formalizing cryptographic security with itrees and proving compiler preservation with rhl. arXiv:2511.11292, 2025. Version 1, 14 November 2025.
- [4] HAETAE Team. HAETAE: Lattice-based digital signature algorithm. Submission to the Korean Post-Quantum Cryptography Competition, 2022. <https://kpqc.or.kr/competition.html>.
- [5] Jasmin contributors. Jasmin language and tool documentation. Language and tool manual distributed with Jasmin, 2026. Consulted from the home-directory Jasmin documentation tree.
- [6] T. F. A. Oliveira. *High-Speed and High-Assurance Cryptographic Software*. PhD thesis, Universidade do Porto, 2022.
- [7] P. Schwabe. Engineering cryptographic software: The jasmin framework. Course slides, Jan. 2026.